# JAVA LAMBDA EXPRESSIONS AND STREAM API

Complete Beginner-to-Advanced Study Guide

Target version: Java 21 | Core introduction: Java 8

Concepts • Definitions • Syntax • Method reference • Stream operations • Collectors • Optional • Practical programs • Interview practice

A structured revision resource for Java learners, backend development and automation engineering preparation.

Prepared as a practical learning reference

# How to use this guide

Study the concepts in order. First understand functional interfaces and lambda syntax; then learn stream pipelines. Do not memorize method names without understanding their input, output and use case.

## Contents


PART I — Lambda Expressions

1. Functional programming and lambda expressions
2. Lambda syntax and variations
3. Functional interfaces
4. Predicate, Function, Consumer and Supplier
5. Other built-in functional interfaces
6. Method references

PART II — Stream API Fundamentals

7. Stream API definition and architecture
8. Creating streams
9. Intermediate operations
10. Terminal operations
11. Lazy evaluation and stream lifecycle

PART III — Advanced Stream API

12. flatMap and mapMulti
13. reduce and collect
14. Collectors class
15. groupingBy and partitioningBy
16. Optional
17. Primitive streams
18. Parallel streams

PART IV — Practical Applications

19. Streams with List, Set and Map
20. Streams with custom objects
21. Employee and student examples
22. Common mistakes and best practices
23. Interview questions and coding exercises

# PART I — LAMBDA EXPRESSIONS

## 1. Functional programming and lambda expressions

Lambda expression — definition: A lambda expression is an anonymous function that provides an implementation for the abstract method of a functional interface. It enables a concise way to pass behavior as data.

Lambda expressions were introduced in Java 8. They are commonly used with functional interfaces, collection operations, event handling, threads and the Stream API.

## Why lambda expressions?

Before Java 8, a small behavior often required an anonymous inner class or a separate implementation class. A lambda expression reduces this boilerplate when the target type is a functional interface.

```
Runnable task = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running task");
    }
};

// Equivalent lambda form:
Runnable lambdaTask = () -> System.out.println("Running task");

new Thread(lambdaTask).start();
```

Key point: A lambda expression does not stand alone with an arbitrary type. Java uses the target functional-interface type to determine the parameter and return types.

## 2. Lambda syntax

```
(parameters) -> expression

(parameters) -> {
    // statements
    return result;
}
```

| Form | Example | Explanation |
|---|---|---|
| No parameters | () -> System.out.println("Hi") | Parentheses are required. |
| One parameter | name -> name.length() | Parentheses may be omitted for one inferred parameter. |
| Multiple parameters | (a, b) -> a + b | Use parentheses for multiple parameters. |
| Explicit types | (int a, int b) -> a + b | Parameter types may be written explicitly. |
| Block body | (a, b) -> { int c = a+b; return c; } | Use braces and return when multiple statements are needed. |
| Expression body | (a, b) -> a + b | A single expression returns its value implicitly. |

## Lambda examples

```
@FunctionalInterface
interface Calculator {
    int operate(int a, int b);
}

public class LambdaDemo {
    public static void main(String[] args) {
        Calculator add = (a, b) -> a + b;
        Calculator multiply = (a, b) -> a * b;

        System.out.println(add.operate(10, 20));       // 30
        System.out.println(multiply.operate(10, 20));  // 200
    }
}
```

## 3. Functional interfaces

Definition: A functional interface is an interface with exactly one abstract method. It is also called a SAM (Single Abstract Method) interface. It may also contain default and static methods.

```
@FunctionalInterface
interface Greeting {
    void greet(String name);

    default void info() {
        System.out.println("Greeting interface");
    }

    static void help() {
        System.out.println("Static helper");
    }
}

public class Main {
    public static void main(String[] args) {
        Greeting g = name -> System.out.println("Hello " + name);
        g.greet("Jaideep");
        g.info();
        Greeting.help();
    }
}
```

Key point: @FunctionalInterface asks the compiler to verify the single-abstract-method rule. Object methods such as toString() do not count as additional abstract methods for this rule.

## 4. Four core built-in functional interfaces

| Interface | Abstract method | Input → output | Typical use |
|---|---|---|---|
| Predicate | test(T) | T → boolean | Check a condition; filtering. |
| Function | apply(T) | T → R | Transform one value into another. |
| Consumer | accept(T) | T → void | Print, log or perform an action. |
| Supplier | get() | none → T | Generate or provide a value. |

## Predicate

Predicate represents a boolean-valued condition on one input. Its main method is test(T).

```
Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(10));  // true

Predicate<Integer> positive = n -> n > 0;
Predicate<Integer> small = n -> n < 100;

System.out.println(positive.and(small).test(50)); // true
System.out.println(positive.or(small).test(-5));  // true
System.out.println(isEven.negate().test(10));     // false
```

Useful methods: test(), and(), or(), negate(), isEqual().

## Function

Function transforms an input value into a result. Its main method is apply(T).

```
Function<String, Integer> length = String::length;
Function<Integer, Integer> square = n -> n * n;

System.out.println(length.apply("Java")); // 4
System.out.println(square.apply(5));      // 25

Function<Integer, Integer> plusTwo = n -> n + 2;
Function<Integer, Integer> timesThree = n -> n * 3;

System.out.println(plusTwo.andThen(timesThree).apply(4)); // 18
System.out.println(plusTwo.compose(timesThree).apply(4)); // 14
```

Useful methods: apply(), andThen(), compose(), identity(). `andThen` applies the current function first; `compose` applies the supplied function first.

## Consumer

```
Consumer<String> printer = value -> System.out.println(value);
printer.accept("Java");

Consumer<String> upperPrinter =
    printer.andThen(value -> System.out.println(value.toUpperCase()));

upperPrinter.accept("stream");
```

Useful methods: accept(), andThen(). A Consumer performs an action and returns no result.

## Supplier

```
Supplier<String> message = () -> "Welcome to Java";
System.out.println(message.get());

Supplier<List<String>> listSupplier = ArrayList::new;
List<String> names = listSupplier.get();
```

Useful method: get(). A Supplier accepts no input and provides a result.

# 5. Other useful built-in functional interfaces

| Interface | Abstract method | Use case |
|---|---|---|
| BiPredicate | test(T,U) | Test a condition using two inputs. |
| BiFunction | apply(T,U) | Combine two inputs into a result. |
| BiConsumer | accept(T,U) | Perform an action using two inputs. |
| UnaryOperator | apply(T) | Function where input and output types are the same. |
| BinaryOperator | apply(T,T) | Combine two values of the same type. |
| IntPredicate / LongPredicate / DoublePredicate | test(primitive) | Avoid wrapper boxing for primitive predicates. |
| IntFunction / LongFunction / DoubleFunction | apply(primitive) | Primitive input to object result. |
| ToIntFunction / ToLongFunction / ToDoubleFunction | applyAsInt/Long/Double | Extract primitive numeric values from objects. |
| IntConsumer / LongConsumer / DoubleConsumer | accept(primitive) | Consume primitive values. |
| IntSupplier / LongSupplier / DoubleSupplier | getAsInt/Long/Double | Supply primitive values. |

```
BiPredicate<String, Integer> longEnough =
    (word, minLength) -> word.length() >= minLength;

BiFunction<Integer, Integer, Integer> add = Integer::sum;
BiConsumer<String, Integer> show =
    (name, age) -> System.out.println(name + " " + age);

UnaryOperator<String> upper = String::toUpperCase;
BinaryOperator<Integer> multiply = (a, b) -> a * b;
```

# 6. Method references

Definition: A method reference is shorthand for a lambda expression that calls an existing method. The `::` operator is used.

| Kind | Syntax | Example |
|---|---|---|
| Static method | ClassName::staticMethod | Integer::parseInt |
| Instance method of a particular object | object::instanceMethod | System.out::println |

| Kind | Syntax | Example |
| --- | --- | --- |
| Instance method of an arbitrary object of a type | ClassName::instanceMethod | String::toUpperCase |
| Constructor reference | ClassName::new | ArrayList::new |

```
List<String> names = List.of("java", "python", "sql");

// Lambda:
names.forEach(name -> System.out.println(name));

// Method reference:
names.forEach(System.out::println);

// Static method reference:
Function<String, Integer> parser = Integer::parseInt;

// Arbitrary object's instance method:
Function<String, String> upper = String::toUpperCase;

// Constructor reference:
Supplier<List<String>> factory = ArrayList::new;
```

Key point: A method reference is not automatically interchangeable with every lambda. Its referenced method must match the functional interface's parameter and return shape.

# PART II — STREAM API FUNDAMENTALS

## 7. Stream API definition and architecture

Definition: The Java Stream API is a set of classes and interfaces for processing sequences of elements from a source using a pipeline of operations. It supports functional-style operations such as filtering, mapping, sorting, reduction and collection.

A Stream is not a data structure and does not store elements. It describes a computation over a source such as a collection, array, generated sequence or I/O resource.

| Collection | Stream |
|---|---|
| Stores elements. | Processes elements from a source. |
| Can generally be traversed repeatedly. | A stream can be consumed only once. |
| Can support adding/removing elements depending on implementation. | Does not add or remove elements from its source. |
| External iteration is common. | Internal iteration is used by stream operations. |

## Stream pipeline

| Stage | Purpose | Examples |
|---|---|---|
| Source | Provides elements. | List.stream(), Arrays.stream(), Stream.of() |
| Intermediate operations | Transform or select elements; return a stream. | filter(), map(), sorted(), distinct() |
| Terminal operation | Triggers execution and produces a result or side effect. | toList(), collect(), count(), reduce(), forEach() |

## 8. Creating streams

```
List<Integer> values = List.of(10, 20, 30);

// From a collection
Stream<Integer> a = values.stream();

// From supplied values
Stream<String> b = Stream.of("Java", "Python", "SQL");

// From an array
int[] nums = {1, 2, 3};
IntStream c = Arrays.stream(nums);

// Empty stream
Stream<String> d = Stream.empty();

// Nullable single value (Java 9+)
Stream<String> e = Stream.ofNullable(null); // empty stream

// Generated stream (usually infinite)
Stream<Double> randoms = Stream.generate(Math::random);

// Iterative stream
Stream<Integer> sequence = Stream.iterate(1, n -> n + 1);

// Bounded iterate (Java 9+)
Stream<Integer> bounded = Stream.iterate(1, n -> n <= 5, n -> n + 1);

// Concatenate two streams
Stream<Integer> combined = Stream.concat(Stream.of(1, 2), Stream.of(3, 4));

// Builder
Stream<String> built = Stream.<String>builder()
    .add("A").add("B").build();
```

Key point: Infinite streams must be bounded when you want a finite result. Use limit() or the predicate overload of iterate().

## 9. Intermediate operations

Intermediate operations return another stream and are generally lazy: they are not evaluated until a terminal operation is invoked.

| Method | Purpose | Typical use case |
|---|---|---|
| filter(Predicate) | Keep matching elements. | Employees with salary > 50,000. |
| map(Function) | Transform each element. | Names to uppercase; object to field. |
| flatMap(Function) | Transform each element to a stream, then flatten. | Flatten list of lists; extract employee skills. |
| mapToInt / mapToLong / mapToDouble | Map to primitive stream. | Extract marks, IDs or prices. |
| flatMapToInt / Long / Double | Flatten into primitive stream. | Flatten nested numeric data. |
| distinct() | Remove duplicates according to equals(). | Unique names or IDs. |
| sorted() | Natural-order sorting. | Sort numbers or strings. |
| sorted(Comparator) | Custom sorting. | Sort employees by salary. |
| peek(Consumer) | Observe elements passing through pipeline. | Debugging. |
| limit(n) | Keep at most n elements. | Top N after sorting. |
| skip(n) | Discard first n elements. | Offset-style pagination. |
| takeWhile(Predicate) | Take initial elements while condition holds. | Take initial passing values in an ordered sequence. |
| dropWhile(Predicate) | Drop initial elements while condition holds. | Ignore initial header/sentinel values. |
| mapMulti() | Emit zero or more results per input. | Expand one record without creating nested streams. |
| unordered() | Remove encounter-order constraint. | Operations where order is irrelevant. |

## filter()

```
List<Integer> even = List.of(1, 2, 3, 4, 5, 6).stream()
    .filter(n -> n % 2 == 0)
    .toList();
// [2, 4, 6]
```

## map()

```
List<String> upper = List.of("java", "sql").stream()
    .map(String::toUpperCase)
    .toList();
// [JAVA, SQL]
```

## map() versus flatMap()

```
List<List<Integer>> nested = List.of(
    List.of(1, 2), List.of(3, 4)
);

// map produces Stream<Stream<Integer>>:
Stream<Stream<Integer>> nestedStreams =
    nested.stream().map(List::stream);

// flatMap produces Stream<Integer>:
List<Integer> flattened = nested.stream()
    .flatMap(List::stream)
    .toList();
// [1, 2, 3, 4]
```

## sorted(), distinct(), limit() and skip()

```
List<Integer> values = List.of(5, 2, 2, 9, 1, 7);

List<Integer> result = values.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .limit(3)
    .toList();
// [5, 2, 1]
```

## takeWhile() and dropWhile() — Java 9+

```
List<Integer> values = List.of(2, 4, 6, 7, 8, 10);

List<Integer> prefix = values.stream()
    .takeWhile(n -> n % 2 == 0)
    .toList();
// [2, 4, 6]

List<Integer> remainder = values.stream()
    .dropWhile(n -> n % 2 == 0)
    .toList();
// [7, 8, 10]
```

These methods act on the initial ordered segment. They are not equivalent to filter(): filter() tests every element; takeWhile()/dropWhile() stop their prefix logic at the first failure.

## mapMulti() — Java 16+

```
List<Integer> expanded = Stream.of(1, 2, 3)
    .<Integer>mapMulti((n, out) -> {
        out.accept(n);
        out.accept(n * 10);
    })
    .toList();
// [1, 10, 2, 20, 3, 30]
```

## 10. Terminal operations

| Method | Result | Use case |
|---|---|---|
| forEach(Consumer) | void | Perform an action for each element. |
| forEachOrdered(Consumer) | void | Perform action in encounter order. |
| toList() | List | Create an unmodifiable list (Java 16+). |
| toArray() | Object[] or A[] | Convert to array. |
| toArray(generator) | A[] | Create typed array. |
| toArray(IntFunction) | A[] | Supply array constructor. |
| collect(Collector) | R | Accumulate into collection or aggregate. |
| count() | long | Count elements. |
| min(Comparator) | Optional | Find minimum. |
| max(Comparator) | Optional | Find maximum. |
| reduce(...) | Optional or T | Combine elements into one result. |
| findFirst() | Optional | Find first element. |
| findAny() | Optional | Find any element. |
| anyMatch(Predicate) | boolean | At least one element matches? |
| allMatch(Predicate) | boolean | Every element matches? |
| noneMatch(Predicate) | boolean | No element matches? |

```
List<Integer> values = List.of(10, 20, 30, 40);

long count = values.stream().count();
```

```
Optional<Integer> firstOver25 = values.stream()
    .filter(n -> n > 25)
    .findFirst();

boolean hasLarge = values.stream().anyMatch(n -> n > 35);
boolean allPositive = values.stream().allMatch(n -> n > 0);
boolean noNegative = values.stream().noneMatch(n -> n < 0);

int sum = values.stream().reduce(0, Integer::sum);

String[] array = values.stream().map(String::valueOf)
    .toArray(String[]::new);
```

## 11. Lazy evaluation and stream lifecycle

Intermediate operations build a pipeline but generally do not process elements immediately. A terminal operation triggers traversal and evaluation. A stream is single-use: after a terminal operation, it cannot be reused.

```
Stream<Integer> stream = Stream.of(1, 2, 3, 4)
    .filter(n -> {
        System.out.println("Testing " + n);
        return n > 2;
    });

// No output yet: no terminal operation.

long count = stream.count(); // Pipeline executes here.

// Do not call stream.count() again: stream is already consumed.
```

Key point: Stream pipelines may short-circuit. For example, findFirst(), anyMatch() and limit() can stop processing once their result is known.

# PART III — ADVANCED STREAM API

## 12. flatMap and mapMulti

flatMap() is useful when each input maps to a stream and the resulting streams must be flattened into one stream.

```
List<List<String>> teams = List.of(
    List.of("Asha", "Ravi"),
    List.of("Mina", "John")
);

List<String> members = teams.stream()
    .flatMap(List::stream)
    .toList();
// [Asha, Ravi, Mina, John]
```

For custom objects:

```
class Employee {
    private final String name;
    private final List<String> skills;

    Employee(String name, List<String> skills) {
        this.name = name;
        this.skills = skills;
    }
    public String getName() { return name; }
    public List<String> getSkills() { return skills; }
}

List<String> allSkills = employees.stream()
    .flatMap(e -> e.getSkills().stream())
    .distinct()
    .toList();
```

Use flatMap when the mapping naturally returns a stream. Consider mapMulti when emitting a small number of results per input and avoiding many short-lived stream objects is useful.

## 13. reduce() and collect()

reduce: Combines elements into one value using an associative operation. It is appropriate for sums, products, min/max and other immutable reductions.

| Overload | Concept | Return |
|---|---|---|
| reduce(BinaryOperator<T>) | No identity; combine elements. | Optional<T> |
| reduce(T identity, BinaryOperator<T>) | Start with identity and combine. | T |
| reduce(U identity, BiFunction<U,? super T,U>, BinaryOperator<U>) | Map/accumulate and combine partial results. | U |

```
List<Integer> values = List.of(1, 2, 3, 4);

Optional<Integer> product = values.stream()
    .reduce((a, b) -> a * b); // Optional[24]

int sum = values.stream()
    .reduce(0, Integer::sum); // 10
```

collect: A mutable reduction operation that accumulates elements into a result container, usually using a Collector.

```
List<String> names = Stream.of("Asha", "Ravi", "Mina")
    .filter(name -> name.length() > 3)
    .collect(Collectors.toList());

Set<String> unique = names.stream()
    .collect(Collectors.toSet());
```

Key point: Do not use reduce() to mutate a shared ArrayList or other mutable container. Use collect() with an appropriate Collector for mutable reduction.

## 14. Collectors class

Collectors is a utility class in java.util.stream. Its static methods create Collector objects passed to Stream.collect().

| Collector method | Purpose / use case |
|---|---|
| toList() | Collect elements into a List; mutability is not guaranteed. |
| toSet() | Collect elements into a Set. |
| toCollection(factory) | Collect into a chosen collection implementation. |
| toUnmodifiableList() / Set() | Create unmodifiable collection (Java 10+). |
| toMap(keyMapper, valueMapper) | Create a Map from elements. |
| toMap(..., mergeFunction) | Resolve duplicate keys with a merge function. |
| toConcurrentMap(...) | Create a concurrent Map. |
| joining() | Concatenate character sequences. |
| counting() | Count elements as Long. |
| summingInt/Long/Double() | Sum mapped numeric values. |
| averagingInt/Long/Double() | Average mapped numeric values. |
| summarizingInt/Long/Double() | Produce count, sum, min, average and max statistics. |
| groupingBy() | Group elements by classification key. |
| groupingByConcurrent() | Concurrent grouping collector. |
| partitioningBy() | Partition elements into true and false groups. |
| mapping() | Apply mapping before downstream collector. |
| flatMapping() | Flatten mapped streams before downstream collector. |
| filtering() | Filter before downstream collection (Java 9+). |
| collectingAndThen() | Apply finishing transformation after collection. |
| reducing() | Reduction as a downstream collector. |
| minBy() / maxBy() | Find minimum or maximum downstream. |
| teeing() | Run two collectors and combine their results (Java 12+). |

## toMap() and duplicate keys

```
List<String> words = List.of("Java", "SQL", "Python", "C");

Map<String, Integer> lengths = words.stream()
    .collect(Collectors.toMap(
        Function.identity(),
        String::length
    ));

// Duplicate keys need a merge function:
Map<Integer, String> byLength = words.stream()
    .collect(Collectors.toMap(
        String::length,
        Function.identity(),
        (oldValue, newValue) -> oldValue + ", " + newValue
    ));
```

Without a merge function, duplicate keys cause IllegalStateException. Choose a merge rule that matches the business requirement; do not silently discard values unless that is intended.

## joining()

```
String csv = Stream.of("Java", "Python", "SQL")
    .collect(Collectors.joining(", "));
// Java, Python, SQL

String bracketed = Stream.of("A", "B", "C")
    .collect(Collectors.joining(" | ", "[", "]"));
// [A | B | C]
```

## 15. groupingBy() and partitioningBy()

```
Map<String, List<Employee>> byDepartment = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

Map<String, Long> departmentCounts = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ));

Map<String, Double> averageSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));

Map<Boolean, List<Student>> passFail = students.stream()
    .collect(Collectors.partitioningBy(s -> s.getMarks() >= 40));
```

| groupingBy() | partitioningBy() |
|---|---|
| Groups by a classifier; may produce many keys. | Splits according to a predicate into true and false keys. |
| Example: department, city, category. | Example: pass/fail, active/inactive. |
| Returns Map<K, ...>. | Returns Map<Boolean, ...>. |

## Downstream collectors: mapping, filtering, maxBy

```
Map<String, List<String>> namesByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));

Map<String, List<Employee>> highEarnersByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.filtering(
            e -> e.getSalary() > 50000,
            Collectors.toList()
        )
    ));

Map<String, Optional<Employee>> topByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));
```

Difference: Stream.filter() removes elements before grouping. Collectors.filtering() filters within each group, so a group can remain present with an empty downstream result.

## 16. Optional

Definition: Optional is a container that may hold a non-null value or be empty. It makes possible absence explicit in an API.

| Method | Use |
|---|---|
| of(value) | Create Optional for a known non-null value; null causes NullPointerException. |
| ofNullable(value) | Create Optional from nullable value. |
| empty() | Create empty Optional. |

| Method | Use |
|---|---|
| isPresent() / isEmpty() | Check presence / absence. |
| get() | Retrieve value; throws NoSuchElementException if empty. |
| orElse(value) | Return value or fallback; fallback expression is evaluated eagerly. |
| orElseGet(supplier) | Return value or lazily obtain fallback. |
| orElseThrow() | Throw if empty. |
| ifPresent(consumer) | Run action if value exists. |
| ifPresentOrElse(action, emptyAction) | Handle both states (Java 9+). |
| map() | Transform value if present. |
| flatMap() | Chain a function that returns Optional. |
| filter() | Keep value only if predicate passes. |
| or(supplier) | Try an alternative Optional (Java 9+). |
| stream() | Convert to zero-or-one-element stream (Java 9+). |

```
Optional<Employee> found = employees.stream()
    .filter(e -> e.getId() == 101)
    .findFirst();

String employeeName = found
    .map(Employee::getName)
    .orElse("Not found");

found.ifPresentOrElse(
    e -> System.out.println(e.getName()),
    () -> System.out.println("Employee not found")
);
```

Key point: Avoid calling get() without checking presence. Prefer map(), ifPresent(), orElse(), orElseGet() and orElseThrow() according to the required behavior.

## orElse() versus orElseGet()

```
String a = Optional.of("Java")
    .orElse(expensiveFallback()); // expensiveFallback is evaluated first

String b = Optional.of("Java")
    .orElseGet(() -> expensiveFallback()); // supplier is not called if present
```

## 17. Primitive streams

IntStream, LongStream and DoubleStream specialize stream processing for primitive numeric values and provide numeric terminal operations.

| Method | Use |
|---|---|
| sum() | Sum values. |
| average() | Return OptionalDouble average. |
| min() / max() | Return OptionalInt, OptionalLong or OptionalDouble. |
| count() | Count values. |
| summaryStatistics() | Return count, sum, min, average and max. |
| range(start, end) | Generate values excluding end (IntStream/LongStream). |
| rangeClosed(start, end) | Generate values including end (IntStream/LongStream). |
| boxed() | Convert primitive values to wrapper-object stream. |
| mapToObj() | Convert primitive values to object stream. |
| asLongStream() / asDoubleStream() | Convert IntStream to wider primitive stream. |

```
int sum = IntStream.rangeClosed(1, 10).sum(); // 55
```

```
double average = IntStream.of(70, 80, 90)
    .average()
    .orElse(0.0); // 80.0

IntSummaryStatistics stats = IntStream.of(10, 20, 30)
    .summaryStatistics();

List<Integer> boxed = IntStream.rangeClosed(1, 5)
    .boxed()
    .toList();
```

## 18. Parallel streams

A parallel stream divides work into subtasks that may be processed concurrently. Collection.parallelStream() creates one; stream.parallel() changes a stream's execution mode.

```
long total = LongStream.rangeClosed(1, 1_000_000)
    .parallel()
    .sum();

List<Integer> ordered = IntStream.rangeClosed(1, 10)
    .parallel()
    .boxed()
    .toList();
```

| Sequential stream | Parallel stream |
|---|---|
| Processes pipeline sequentially. | May process portions concurrently. |
| Usually simpler and predictable. | May improve throughput for suitable CPU-bound work. |
| Encounter order is easier to reason about. | forEach() output order may differ; use forEachOrdered() when needed. |
| No parallel splitting overhead. | Splitting, coordination and merging add overhead. |

Key point: Parallel is not synonymous with faster. Avoid shared mutable state, benchmark realistic workloads, and be cautious with blocking I/O and order-sensitive operations.

## BaseStream methods

| Method | Purpose |
|---|---|
| iterator() | Obtain iterator for manual traversal. |
| spliterator() | Obtain spliterator for traversal/splitting. |
| isParallel() | Check parallel mode. |
| sequential() | Switch to sequential mode. |
| parallel() | Switch to parallel mode. |
| unordered() | Remove encounter-order constraint. |
| onClose(Runnable) | Register a close handler. |
| close() | Close stream and run close handlers. |

Close streams that wrap closeable I/O resources, such as Files.lines(). Ordinary collection streams usually do not need explicit closing.

# PART IV — PRACTICAL APPLICATIONS

## 19. Streams with List, Set and Map

List and Set expose stream() and parallelStream(). Map does not directly expose stream(); stream its keySet(), values() or entrySet().

```
List<String> names = List.of("Asha", "Ravi", "Mina");
names.stream().filter(n -> n.startsWith("A")).toList();

Set<String> unique = new HashSet<>(names);
unique.stream().map(String::toUpperCase).forEach(System.out::println);

Map<Integer, String> students = Map.of(
    101, "Asha", 102, "Ravi", 103, "Mina"
);

students.keySet().stream().forEach(System.out::println);
students.values().stream().forEach(System.out::println);

students.entrySet().stream()
    .filter(e -> e.getKey() >= 102)
    .forEach(e -> System.out.println(e.getKey() + " : " + e.getValue()));

Map<Integer, String> selected = students.entrySet().stream()
    .filter(e -> e.getKey() >= 102)
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
```

## 20. Streams with custom objects

The usual pattern is to create a domain class with fields and accessors, create a collection of objects, then use method references to extract fields and lambdas for conditions.

```
class Employee {
    private final int id;
    private final String name;
    private final String department;
    private final double salary;

    Employee(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    @Override
    public String toString() {
        return id + " " + name + " " + department + " " + salary;
    }
}
```

## 21. Employee management example

```
List<Employee> employees = List.of(
    new Employee(101, "Jaideep", "IT", 60000),
    new Employee(102, "Rahul", "HR", 40000),
    new Employee(103, "Priya", "IT", 75000),
    new Employee(104, "Arjun", "Finance", 50000),
    new Employee(105, "Sneha", "HR", 55000),
    new Employee(106, "Kiran", "Finance", 80000)
);

// Employees earning more than 50,000
List<Employee> highEarners = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .toList();

// Extract names
List<String> names = employees.stream()
    .map(Employee::getName)
    .toList();

// Salary ascending / descending
List<Employee> ascending = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary))
    .toList();

List<Employee> descending = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .toList();

// Highest-paid employee
Optional<Employee> highest = employees.stream()
    .max(Comparator.comparingDouble(Employee::getSalary));

// Total and average salary
double totalSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .sum();

double averageSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0.0);

// Group by department
Map<String, List<Employee>> byDepartment = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Count by department
Map<String, Long> countByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ));

// Average salary by department
Map<String, Double> averageByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));

// Highest-paid employee in each department
Map<String, Optional<Employee>> highestByDepartment =
    employees.stream().collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));

// Second-highest distinct salary
Optional<Double> secondHighestSalary = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();

// Convert to map keyed by employee ID
Map<Integer, Employee> byId = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));
```

## Student management example

```
class Student {
    private final int id;
    private final String name;
    private final int marks;

    Student(int id, String name, int marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public int getMarks() { return marks; }
}

List<Student> students = List.of(
    new Student(1, "Rahul", 85),
    new Student(2, "Priya", 35),
    new Student(3, "Jaideep", 92),
    new Student(4, "Arjun", 65),
    new Student(5, "Sneha", 30)
);

List<Student> passed = students.stream()
    .filter(s -> s.getMarks() >= 40)
    .toList();

Optional<Student> topper = students.stream()
    .max(Comparator.comparingInt(Student::getMarks));

double averageMarks = students.stream()
    .mapToInt(Student::getMarks)
    .average()
    .orElse(0.0);

Map<Boolean, List<Student>> passFail = students.stream()
    .collect(Collectors.partitioningBy(s -> s.getMarks() >= 40));
```

## E-commerce product examples

```
class Product {
    private final int id;
    private final String name;
    private final String category;
    private final double price;

    Product(int id, String name, String category, double price) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.price = price;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getCategory() { return category; }
    public double getPrice() { return price; }
}

List<Product> expensive = products.stream()
    .filter(p -> p.getPrice() > 1000)
    .toList();

Map<String, Double> averagePrice = products.stream()
    .collect(Collectors.groupingBy(
        Product::getCategory,
        Collectors.averagingDouble(Product::getPrice)
    ));

Map<Integer, Product> productById = products.stream()
    .collect(Collectors.toMap(Product::getId, Function.identity()));
```

## 22. Common mistakes and best practices

| Mistake | Why it is wrong | Better approach |
|---|---|---|
| Reusing a stream after a terminal operation | A stream is single-use. | Create a fresh stream from the source. |
| Expecting intermediate operations to run immediately | They are lazy. | Add a terminal operation. |
| Using get() on Optional without checking | Can throw NoSuchElementException. | Use orElse, orElseGet, ifPresent or orElseThrow. |
| Assuming parallel is always faster | Overhead can exceed benefit. | Benchmark representative workloads. |
| Mutating shared ArrayList in parallel forEach | Data race / unsafe mutation. | Use collect() or toList(). |
| Using reduce() to build mutable containers | Reduction contract can be violated. | Use collect() for mutable reduction. |
| Assuming Collectors.toList() is mutable | Mutability is not guaranteed by its contract. | Use toCollection(ArrayList::new) if mutation is required. |
| Ignoring duplicate keys in toMap() | Duplicate mapped keys throw by default. | Supply a merge function. |
| Using peek() for business logic | It is intended mainly for observation/debugging. | Use map() for transformation or forEach() for terminal actions. |
| Ignoring ordering requirements | Parallel or unordered operations may not preserve order. | Use ordered streams and forEachOrdered() when required. |

## 23. Interview questions

1. What is a lambda expression?
An anonymous function that implements the abstract method of a target functional interface.

2. What is a functional interface?
An interface with exactly one abstract method; default and static methods are allowed.

3. What is the difference between Predicate and Function?
Predicate returns boolean; Function transforms an input into a result.

4. What is the difference between map() and flatMap()?
map transforms each element; flatMap transforms each element into a stream and flattens the results.

5. What is lazy evaluation?
Intermediate operations are not evaluated until a terminal operation triggers the pipeline.

6. Can a stream be reused?
No. A stream is single-use after a terminal operation.

7. What is the difference between collect() and reduce()?
reduce combines values into one result; collect performs mutable reduction into a result container using a Collector.

8. What is the difference between groupingBy and partitioningBy?
groupingBy classifies into potentially many groups; partitioningBy splits into true and false groups.

9. Why does findFirst() return Optional?
The stream may be empty, so there may be no first element.

10. What is the difference between Stream<Integer> and IntStream?
Stream<Integer> processes wrapper objects; IntStream processes primitive int values.

11. What is the difference between toList() and Collectors.toList()?
Stream.toList() returns an unmodifiable list; Collectors.toList() does not guarantee mutability.

12. What is the difference between forEach and forEachOrdered?
forEach does not guarantee encounter order in parallel; forEachOrdered preserves it.

13. What is a parallel stream?
A stream whose pipeline may process portions concurrently.

14. What happens when toMap sees duplicate keys?
The two-argument toMap collector throws IllegalStateException; use a merge function.

15. What is mapMulti?
An intermediate operation that emits zero or more output elements for each input without requiring a nested stream per input.

## Coding practice problems

- 1. Find all even numbers in a list.
- 2. Convert all strings in a list to uppercase.
- 3. Remove duplicate values while preserving encounter order.
- 4. Find the sum and average of a list of integers.
- 5. Find the second-highest distinct number.
- 6. Count the frequency of each word.
- 7. Group strings by their length.
- 8. Find common values between two lists.
- 9. Find the first non-repeated character in a string while preserving character order.
- 10. Group employees by department.
- 11. Find the highest-paid employee in each department.
- 12. Find the top three highest-paid employees.
- 13. Partition students into pass and fail groups.
- 14. Find departments containing more than two employees.
- 15. Extract all unique skills from all employees using flatMap().

## Assignment: Employee data processing

Create an Employee class with id, name, department and salary. Add at least ten employees and implement all tasks below without copying the examples.

- Filter employees earning more than a chosen salary.
- Sort employees by salary descending.
- Find the highest and second-highest distinct salary.
- Calculate total and average salary.
- Group employees by department.
- Count employees in each department.
- Find the highest-paid employee in each department.
- Extract unique department names.
- Convert the employee list to a Map keyed by employee ID.
- Partition employees into salary-above-threshold and salary-at-or-below-threshold groups.

# Appendix A — Java version notes

| Java version | Relevant additions covered in this guide |
|---|---|
| Java 8 | Lambda expressions, functional interfaces, Stream API, Optional, core Collectors, method references. |
| Java 9 | takeWhile(), dropWhile(), Stream.ofNullable(), three-argument iterate(), Optional.stream(), Optional.ifPresentOrElse(), Collectors.filtering(), Collectors.flatMapping(). |
| Java 10 | Collectors.toUnmodifiableList(), toUnmodifiableSet(), toUnmodifiableMap(). |
| Java 12 | Collectors.teeing(). |
| Java 16 | Stream.toList(), mapMulti() and primitive mapMulti variants. |
| Java 21 | All above APIs are available. Java 21 does not add a major new method to the core Stream interface. |

# Appendix B — Essential imports

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;
```

For file-based stream examples, also review java.nio.file.Files and java.nio.file.Path. Streams created from I/O resources should be closed, commonly with try-with-resources.

# Final learning advice

Do not measure progress by how many methods you can recite. Measure it by whether you can look at a new data-processing requirement, choose an appropriate operation, explain the return type, handle empty results and write a correct pipeline.

Key point: Recommended mastery order: lambda syntax → functional interfaces → map/filter → terminal operations → flatMap → Collectors → grouping/partitioning → Optional → primitive streams → parallel streams → custom-object problems.